



 Devops
Shack

Git Scenario Based Q and A

BY DEVOPS SHACK

200 Git Scenario-Based Q&A

1. Scenario: Undo Last Commit but Keep Changes

Problem: You committed too early and want to modify the content, but don't want to lose changes.

Solution:

```
git reset --soft HEAD~1
```

Explanation:

- This moves HEAD back one commit, but leaves your changes in the staging area.
- Ideal for re-editing files or combining changes before recommitting.

Real-World Tip:

Used in CI pipelines when commit hooks fail due to format/style issues.

2. Scenario: Revert a Commit in the Middle of History

Problem: You want to undo the effects of a commit, not delete history.

Solution:

```
git revert <commit-hash>
```

Explanation:

- Creates a new commit that undoes changes from the specified commit.
- Keeps history intact — very useful in production environments.

Real-World Tip:

Often used to roll back a buggy feature without disrupting team history.

3. Scenario: Delete a Local Commit Before Pushing

Problem: You made 2 wrong commits locally. Haven't pushed yet.

Solution:

```
git reset --hard HEAD~2
```

Explanation:

- Resets to two commits before current HEAD.
- Use --hard only if you're sure you don't need those files.



Risk:

Irrecoverable unless you know how to use git reflog.

4. Scenario: Accidentally Deleted Files but Not Committed

Problem: You used `rm *.js` and want to recover uncommitted deletions.

Solution:

```
git checkout -- .
```

Explanation:

- Discards all uncommitted changes in the working directory.
 - Recovers files if not staged/deleted in commit.
-

5. Scenario: Restore a File from a Specific Commit

Problem: Need to bring back an older version of a single file.

Solution:

```
git checkout <commit-hash> -- path/to/file
```

Explanation:

- Only fetches that file from the older commit.
 - Useful for partial reverts or fetching config values.
-

6. Scenario: See Who Last Changed a Line

Problem: You need to find who broke a line in code.

Solution:

```
git blame filename
```

Explanation:

- Annotates each line with the author, date, and commit hash.
 - Pinpoints responsibility — helpful in debugging or code audits.
-

7. Scenario: Rewriting Commit Message Before Push

Problem: You typed the wrong commit message.

Solution:

```
git commit --amend
```

Explanation:

- Opens the commit editor and lets you update message and staged content.
 - Avoid using it after pushing unless you force-push.
-

8. Scenario: Clean Untracked Files and Directories

Problem: Your repo is messy with leftover files.

Solution:

```
git clean -fd
```

Explanation:

- -f forces cleaning.
 - -d includes directories.
 - Dangerous but effective for cleanups in build processes.
-

9. Scenario: Restore a Deleted Branch

Problem: You deleted a branch that wasn't merged.

Solution:

```
git reflog
```

```
git checkout -b branch-name <commit-id>
```

Explanation:

- reflog shows all recent HEAD movements, including deletes.
 - You can create a new branch from the lost commit.
-

10. Scenario: Push a Local Branch to Remote for First Time

Problem: You made a new branch but forgot to push it.

Solution:



```
git push -u origin feature/new-branch
```

Explanation:

- -u sets the upstream so git push and git pull work by default in future.
-

11. Scenario: Work on Two Features Simultaneously

Problem: You're halfway through one feature, but need to pause and start another.

Solution:

```
git stash push -m "work in progress"
```

```
git checkout -b new-feature
```

Explanation:

- stash saves uncommitted work.
 - Perfect for context-switching without polluting branches.
-

12. Scenario: Merge a Feature but Remove History

Problem: Want to squash feature commits before merging to main.

Solution:

```
git merge --squash feature/login
```

```
git commit -m "Add login feature"
```

Explanation:

- Combines all feature commits into a single one before merge.
 - Maintains clean main history.
-

13. Scenario: Track a Remote Branch Locally

Problem: A teammate pushed a branch you want to track.

Solution:

```
git fetch
```

```
git checkout --track origin/feature-x
```

Explanation:



- --track sets up local branch tracking remote.
-

14. Scenario: Fix 'You are not currently on a branch'

Problem: You're in detached HEAD state after checking out a commit.

Solution:

```
git checkout -b temp-fix
```

Explanation:

- Creates a new branch from detached HEAD.
 - Lets you keep work and resume normally.
-

15. Scenario: View Only Commits by a Specific Author

Problem: You want to audit changes by one developer.

Solution:

```
git log --author="Aditya"
```

Explanation:

- Filters commit history for that author.
-

16. Scenario: Save a Patch to Apply Later

Problem: You want to share uncommitted changes via patch.

Solution:

```
git diff > patch.diff
```

Apply with:

```
git apply patch.diff
```

Explanation:

- diff captures changes.
 - apply replays them elsewhere.
-

17. Scenario: Track Down Which Commit Introduced a Bug



Problem: The app started crashing after recent changes.

Solution:

```
git bisect start
```

```
git bisect bad
```

```
git bisect good <known-good-commit>
```

Explanation:

- Binary search algorithm to identify problematic commit.
 - Vital for debugging in legacy or large codebases.
-

18. Scenario: Fix 'fatal: remote origin already exists'

Problem: You tried to add a remote that already exists.

Solution:

```
git remote set-url origin https://github.com/user/repo.git
```

Explanation:

- Updates the URL without removing existing configuration.
-

19. Scenario: Fix 'error: failed to push some refs'

Problem: You tried to push but got a non-fast-forward error.

Solution:

```
git pull --rebase origin main
```

```
git push
```

Explanation:

- Rebase aligns your local changes on top of the remote.
 - Avoids merge commits and fixes push error.
-

20. Scenario: Rename a Local Branch

Problem: You named a branch incorrectly.

Solution:



```
git branch -m new-name
```

Explanation:

- -m allows branch renaming.
- If you're on the branch, it renames the current one.

21. Scenario: Rename a Remote Branch

Problem: You pushed a wrongly named branch and want to rename it on the remote.

Solution:

```
# Rename locally
```

```
git branch -m old-name new-name
```

```
# Delete old remote branch
```

```
git push origin --delete old-name
```

```
# Push new branch and set upstream
```

```
git push -u origin new-name
```

Explanation:

- Git doesn't support direct renaming on remote. You must delete and push again.
 - -u ensures local and remote are in sync for future pulls.
-

22. Scenario: Cherry-Pick Specific Commits to Another Branch

Problem: You want to bring only one feature commit from feature-a into release.

Solution:

```
git checkout release
```

```
git cherry-pick <commit-hash>
```

Explanation:

- Isolates a single commit and re-applies it on a different branch.
 - Ideal for backporting a bug fix to production.
-

23. Scenario: Staging Portions of a File (Hunk)

Problem: You made multiple changes in one file, but want to commit only part of it.

Solution:

```
git add -p filename
```

Explanation:

- -p lets you stage changes interactively, hunk-by-hunk.
 - Crucial for clean, meaningful commits in collaborative teams.
-

24. Scenario: See Unmerged Branches

Problem: You want to list branches that haven't been merged into main.

Solution:

```
git branch --no-merged main
```

Explanation:

- Helps identify stale feature branches or pending reviews.
-

25. Scenario: Switch to a Branch You Just Deleted

Problem: You deleted a branch and forgot you needed it.

Solution:

```
git reflog
```

```
git checkout -b restored-branch <commit-id>
```

Explanation:

- reflog tracks every HEAD movement, even deletes.
 - You can recover nearly anything if you act quickly.
-

26. Scenario: Undo Git Add for a File

Problem: You accidentally staged a file with git add.

Solution:

```
git reset filename
```

Explanation:



- Moves it back to working directory from staging area.
 - Leaves file changes untouched.
-

27. Scenario: Unstage All Files

Problem: You staged a large number of files but want to reset them.

Solution:

```
git reset
```

Explanation:

- Equivalent to removing all files from staging area.
 - Files remain modified; nothing is lost.
-

28. Scenario: Stop Tracking a File but Keep It Locally

Problem: You want to remove a file from Git but keep it on disk.

Solution:

```
git rm --cached filename
```

Explanation:

- Removes file from Git index but not from working directory.
 - Useful for .env, IDE files, or configs that should not be versioned.
-

29. Scenario: Fix Wrong Files in Previous Commit

Problem: You committed the wrong files by mistake.

Solution:

```
# Stage the correct ones
```

```
git add correct-file1 correct-file2
```

```
# Amend previous commit
```

```
git commit --amend
```

Explanation:



- Replaces the content of the last commit without changing the commit history unless already pushed.
-

30. Scenario: Fetch Remote Changes Without Merging

Problem: You want to review what's changed before merging.

Solution:

```
git fetch origin
```

```
git log HEAD..origin/main
```

Explanation:

- fetch downloads but doesn't merge.
 - You can inspect the commits before deciding to merge or rebase.
-

31. Scenario: Compare Your Branch with Another

Problem: You want to see differences between feature-a and main.

Solution:

```
git diff feature-a..main
```

Explanation:

- Shows code difference between branches.
 - Useful for PR reviews and rebases.
-

32. Scenario: See What Files Changed Between Two Commits

Problem: You want a list of changed files between two SHAs.

Solution:

```
git diff --name-only <commit1> <commit2>
```

Explanation:

- Helpful during release planning and testing phases.
-



33. Scenario: Clone a Repo Without History

Problem: You want only the latest state of the project.

Solution:

```
git clone --depth 1 <repo-url>
```

Explanation:

- Performs a shallow clone.
 - Useful in CI jobs where history isn't needed.
-

34. Scenario: Sync Fork with Upstream

Problem: Your fork is outdated. You want to update it from the original repo.

Solution:

```
git remote add upstream https://github.com/original/repo.git
```

```
git fetch upstream
```

```
git merge upstream/main
```

Explanation:

- Merges upstream changes into your forked main.
-

35. Scenario: Reset a Single File to Remote State

Problem: You broke a file and want the latest from origin/main.

Solution:

```
git fetch origin
```

```
git checkout origin/main -- path/to/file
```

Explanation:

- Pulls only that file from the remote and replaces local version.
-

36. Scenario: Completely Remove a File from Git History

Problem: A password or secret key was committed. You want to delete it from all history.



Solution:

```
git filter-branch --force --index-filter \  
"git rm --cached --ignore-unmatch path/to/secret.file" \  
--prune-empty --tag-name-filter cat -- --all
```

Explanation:

- Rewrites entire history. Dangerous but powerful.
 - Use BFG Repo Cleaner for easier usage.
-

37. Scenario: Clone a Single Folder from a Large Repo

Problem: You want only the docs/ folder from a massive repo.

Solution:

```
git clone --depth 1 --filter=blob:none --sparse <repo>  
cd <repo>  
git sparse-checkout set docs
```

Explanation:

- Sparse checkout allows partial clone. Saves bandwidth and time.
-

38. Scenario: Switch Base Branch of a Feature

Problem: Your branch is based on dev but now needs to be on main.

Solution:

```
git rebase --onto main dev feature-branch
```

Explanation:

- Rebases feature-branch onto main, dropping the dev base.
 - Avoids redoing changes manually.
-

39. Scenario: Save Work in Progress Across Machines

Problem: You want to continue your work from another machine without pushing to origin.



Solution:

```
git bundle create work.bundle --all
```

```
# On other machine:
```

```
git clone work.bundle repo-folder
```

Explanation:

- Bundles repo into a single file. Used for offline sharing or backups.
-

40. Scenario: Find Commits That Touched a File

Problem: You want to review every change made to main.py.

Solution:

```
git log --follow main.py
```

Explanation:

- Tracks renames and shows full commit history of a file.

41. Scenario: Show File Diffs in a Commit

Problem: You want to see what changed inside a commit.

Solution:

```
git show <commit-hash>
```

Explanation:

- Displays the complete patch with diffs, authorship, and commit message.
 - Best for code reviews or understanding what a commit introduced.
-

42. Scenario: Revert a Merge Commit

Problem: You merged a branch but want to undo the merge commit itself.

Solution:

```
git revert -m 1 <merge-commit-hash>
```

Explanation:

- -m 1 selects the parent to keep. In most cases, parent 1 is the base (e.g., main).
- Essential for undoing a merge while keeping history clean.



43. Scenario: Apply Stash Without Removing It

Problem: You want to apply stashed changes but keep them for later use.

Solution:

```
git stash apply
```

Explanation:

- Applies the stash but doesn't delete it.
 - Good for experimenting with changes without losing stash.
-

44. Scenario: Recover Lost Commits After Hard Reset

Problem: You used `git reset --hard` and lost important commits.

Solution:

```
git reflog
```

```
git checkout <lost-commit-hash>
```

Explanation:

- Reflog tracks all HEAD movements — even deleted branches or resets.
 - Always your first stop in accidental loss scenarios.
-

45. Scenario: Temporarily Ignore Local Changes

Problem: You need to pull the latest changes but have local edits you don't want to commit yet.

Solution:

```
git stash push -m "WIP before pull"
```

```
git pull
```

```
git stash pop
```

Explanation:

- stash saves your state, pull updates repo, and pop re-applies your changes.
- Safe and commonly used before rebases or merges.



46. Scenario: Display a Visual Branch Tree

Problem: You want to see how commits and branches diverge visually.

Solution:

```
git log --oneline --graph --all --decorate
```

Explanation:

- Visualizes commit history as a tree.
 - Useful for understanding complex histories, merges, and rebase effects.
-

47. Scenario: Commit a Fix Without Pulling First

Problem: The remote changed, but you need to quickly push a fix without pulling.

Solution:

```
git commit ...
```

```
git pull --rebase
```

```
git push
```

Explanation:

- Rebase integrates changes without merge commits.
 - Safer than git push --force for shared branches.
-

48. Scenario: Create a Patch from a Commit

Problem: You want to email/share a fix as a .patch file.

Solution:

```
git format-patch -1 <commit-hash>
```

Explanation:

- Creates a patch file with metadata and diffs.
 - Ideal for sending code through email or Gerrit-style reviews.
-

49. Scenario: Stage New Files But Not Deleted Ones



Problem: You want to stage only new files, not deletions.

Solution:

```
git add --intent-to-add .
```

Explanation:

- This adds new files while skipping deletions.
 - You can review deletions manually later.
-

50. Scenario: Temporarily Change Git Author for One Commit

Problem: You're committing on behalf of a teammate once.

Solution:

```
GIT_AUTHOR_NAME="John Doe" GIT_AUTHOR_EMAIL="john@example.com" git commit -m "Hotfix"
```

Explanation:

- Environment variables override global gitconfig.
 - Used when teammates share machines or commits.
-

51. Scenario: Fix a File Committed with Wrong Permissions

Problem: You pushed a file with executable permission by mistake.

Solution:

```
chmod -x file.sh
```

```
git add file.sh
```

```
git commit --amend
```

Explanation:

- Git tracks permission bits. This fixes accidental +x flags.
-

52. Scenario: Clean Local Tags Not on Remote

Problem: Your local tags are cluttered and inconsistent with remote.

Solution:



```
git tag -l | xargs git tag -d
```

```
git fetch --tags
```

Explanation:

- Deletes all local tags and re-fetches from the remote.
-

53. Scenario: Create a Tag and Push It

Problem: You want to mark a version release.

Solution:

```
git tag -a v2.0 -m "Version 2.0 release"
```

```
git push origin v2.0
```

Explanation:

- Annotated tags include metadata like author, date, and message.
-

54. Scenario: Clean Up All Merged Branches

Problem: You have too many merged feature branches.

Solution:

```
git branch --merged main | grep -v '^*' | xargs -n 1 git branch -d
```

Explanation:

- Lists merged branches and deletes them.
 - Keeps workspace tidy after successful PRs.
-

55. Scenario: Find Unreachable Commits

Problem: You suspect that there are commits not reachable from any branch.

Solution:

```
git fsck --unreachable
```

Explanation:

- Shows commits that aren't referenced anymore.
- Helps catch things lost due to resets or rebases.



56. Scenario: Squash Last N Commits into One

Problem: You want to collapse 3 commits into a single one.

Solution:

```
git reset --soft HEAD~3
```

```
git commit -m "Combine changes"
```

Explanation:

- --soft retains staging area.
 - Use before push, especially in code cleanup.
-

57. Scenario: Push a Commit to Another Remote

Problem: You maintain two remotes: origin and staging.

Solution:

```
git push staging main
```

Explanation:

- You can configure and push to any remote using Git's flexibility.
-

58. Scenario: Fix 'warning: Pulling without specifying how to reconcile divergent branches'

Problem: Git warns you during git pull.

Solution:

```
git config pull.rebase true
```

Explanation:

- Sets pull behavior globally to rebase instead of merge.
 - Helps keep history linear.
-

59. Scenario: Keep Local Changes While Switching Branches

Problem: You tried to checkout but got blocked by local changes.



Solution:

git stash

git checkout new-branch

git stash pop

Explanation:

- Safely stores uncommitted changes so you can switch branches.
-

60. Scenario: Make a Branch from a Tag

Problem: You want to work on a release from v1.0.

Solution:

git checkout -b release-v1 v1.0

Explanation:

- Creates a new branch pointing to the tag.
- Used in hotfixes or minor patch releases based on a previous tag.

61. Scenario: Avoid Committing Secrets

Problem: You accidentally committed an .env file with API keys.

Solution:

git rm --cached .env

echo ".env" >> .gitignore

git commit -m "Remove .env from tracking"

Explanation:

- Removes the file from Git while keeping it locally.
 - Prevents future commits by adding it to .gitignore.
-

62. Scenario: Verify Commit Signature

Problem: You want to check if a commit is signed and verified.



Solution:

```
git log --show-signature
```

Explanation:

- Verifies GPG-signed commits.
 - Essential in regulated or enterprise environments.
-

63. Scenario: Disable Git Auto-CRLF Line Ending Conversions

Problem: Git is modifying line endings, breaking scripts on Linux.

Solution:

```
git config --global core.autocrlf false
```

Explanation:

- Prevents Git from changing line endings.
 - Crucial when collaborating across Windows/Linux environments.
-

64. Scenario: Use Git Over SSH Instead of HTTPS

Problem: You're tired of entering your GitHub credentials repeatedly.

Solution:

```
git remote set-url origin git@github.com:user/repo.git
```

Explanation:

- SSH keys automate authentication.
 - Faster, more secure for developers working daily.
-

65. Scenario: Create a Branch from a Past Commit

Problem: You want to experiment starting from an old commit.

Solution:

```
git checkout -b feature/experiment <old-commit-hash>
```

Explanation:

- Creates a new branch without affecting existing history.



- Used for testing or re-implementing an approach.
-

66. Scenario: Permanently Ignore a File Across All Repos

Problem: You want Git to ignore OS-generated files like .DS_Store everywhere.

Solution:

```
echo ".DS_Store" >> ~/.gitignore_global
```

```
git config --global core.excludesfile ~/.gitignore_global
```

Explanation:

- Applies globally to all repos on your system.
 - Useful for ignoring system artifacts.
-

67. Scenario: Show Changes Between Local and Remote

Problem: You want to check what you're about to push.

Solution:

```
git log origin/main..HEAD
```

Explanation:

- Displays commits present locally but not yet pushed.
 - Helps ensure nothing is accidentally released.
-

68. Scenario: Add a Remote to a Cloned Repo

Problem: You cloned your own repo and now want to track upstream changes.

Solution:

```
git remote add upstream https://github.com/original/repo.git
```

Explanation:

- You can now fetch or pull from upstream to sync changes.
-

69. Scenario: Resolve Conflict Without Keeping Any Changes

Problem: You hit a merge conflict and want to discard your side entirely.



Solution:

```
git checkout --theirs conflicted-file
```

```
git add conflicted-file
```

Explanation:

- --theirs favors the incoming changes.
 - Ideal when you're confident the other branch is correct.
-

70. Scenario: Track a File Again After Removing from Git

Problem: You removed a file with --cached and now want to track it again.

Solution:

```
git add filename
```

```
git commit -m "Track file again"
```

Explanation:

- Simply add it back; Git resumes tracking.
-

71. Scenario: Set Default Branch to Main Instead of Master

Problem: Your org uses main but new repos default to master.

Solution:

```
git config --global init.defaultBranch main
```

Explanation:

- Ensures consistency when initializing repos.
-

72. Scenario: List All Commits That Touched a Directory

Problem: You want to see who modified anything in src/.

Solution:

```
git log -- src/
```

Explanation:

- Restricts log to changes inside that folder.



- Perfect for ownership and blame audits.
-

73. Scenario: See Number of Commits per Author

Problem: You want commit statistics by contributor.

Solution:

```
git shortlog -sn
```

Explanation:

- Shows commit count per author.
 - Good for team analytics or dashboards.
-

74. Scenario: View All Tags and Associated Commits

Problem: You want to see a list of all tags and what they point to.

Solution:

```
git show-ref --tags
```

Explanation:

- Displays full SHA and tag reference.
 - Essential for version traceability.
-

75. Scenario: Make an Empty Commit (No Changes)

Problem: You want to trigger CI/CD or mark a milestone.

Solution:

```
git commit --allow-empty -m "Trigger CI for deployment"
```

Explanation:

- Creates a commit without any changes.
 - Useful in automations or fixing blocked pipelines.
-

76. Scenario: Fetch Only Specific Branch

Problem: You want to clone just release-v2 from a large repo.



Solution:

```
git clone --branch release-v2 --single-branch <repo-url>
```

Explanation:

- Optimizes clone size.
 - Ideal for build servers or restricted access use cases.
-

77. Scenario: Track Changes to a Binary File

Problem: You want to store .png or .jar files but not bloat the repo.

Solution:

Use **Git LFS** (Large File Storage):

```
git lfs install
```

```
git lfs track "*.png"
```

```
git add .gitattributes
```

```
git add file.png
```

Explanation:

- Offloads large files to external storage.
 - Maintains performance in clone and fetch.
-

78. Scenario: Undo Only Last Commit Message

Problem: You want to change the last commit message but not the code.

Solution:

```
git commit --amend -m "New message only"
```

Explanation:

- Keeps all content the same.
 - Updates the message only.
-

79. Scenario: Stop Tracking a Folder

Problem: You want to remove logs/ from Git tracking.



Solution:

```
git rm -r --cached logs/  
echo "logs/" >> .gitignore  
git commit -m "Stop tracking logs folder"
```

Explanation:

- Useful when a folder becomes a generated artifact post-build.
-

80. Scenario: Copy Changes from One Branch Without Switching

Problem: You're on main but want a file from feature-a.

Solution:

```
git checkout feature-a -- path/to/file
```

Explanation:

- Grabs the file from another branch and leaves you on current branch.
- Useful for selective code reuse.

81. Scenario: Fix a Detached HEAD After Checking Out a Commit

Problem: You checked out a specific commit and now can't commit further.

Solution:

```
git checkout -b hotfix-from-commit
```

Explanation:

- Detached HEAD means you're not on any branch.
 - Creating a new branch from that commit lets you preserve your work and push changes later.
-

82. Scenario: Remove All Files Except .git Directory

Problem: You want to clean your working directory but preserve Git tracking.

Solution:

```
find . -not -path './.git*' -delete
```



Explanation:

- Removes everything except .git.
 - You can now reset the repo to any commit cleanly.
-

83. Scenario: Prevent Git from Tracking File Mode (chmod)

Problem: Git keeps detecting permission changes as diffs.

Solution:

```
git config core.fileMode false
```

Explanation:

- Ignores file permission changes.
 - Common in team setups across Windows/Linux.
-

84. Scenario: See When a Line Was Introduced

Problem: You want to know which commit added a line.

Solution:

```
git blame -L <start>,<end> filename
```

Explanation:

- -L restricts blame to specific lines.
 - Used in debugging or auditing legacy systems.
-

85. Scenario: Remove a Specific Commit from History

Problem: A specific commit (not HEAD) needs to be erased completely.

Solution:

```
git rebase -i <commit~1>
```

In editor: replace `pick` with `drop` for that commit

Explanation:

- rebase -i allows you to surgically modify history.
- Only safe before pushing or with force-push coordination.



86. Scenario: Push to Multiple Remotes Simultaneously

Problem: You want to push code to both GitHub and GitLab.

Solution:

```
git remote set-url --add --push origin git@github.com:user/repo.git
```

```
git remote set-url --add --push origin git@gitlab.com:user/repo.git
```

```
git push
```

Explanation:

- You can push to multiple URLs defined under the same remote.
-

87. Scenario: Abort an In-Progress Merge

Problem: Merge is in progress and you want to cancel it completely.

Solution:

```
git merge --abort
```

Explanation:

- Safely reverts your repo to pre-merge state.
 - Preferred over reset when in the middle of a merge.
-

88. Scenario: Abort an In-Progress Rebase

Problem: Rebase is stuck in a conflict and you want to cancel it.

Solution:

```
git rebase --abort
```

Explanation:

- Rolls back everything done during the rebase session.
-

89. Scenario: Merge Without Fast-Forward

Problem: You want to preserve merge commits for visibility.



Solution:

```
git merge --no-ff feature-branch
```

Explanation:

- Creates a merge commit even if Git could fast-forward.
 - Maintains clear history with visible branches.
-

90. Scenario: Detect if a Branch Has Diverged

Problem: You're unsure whether your local branch differs from remote.

Solution:

```
git fetch
```

```
git status
```

Explanation:

- Git will tell you if your branch is ahead/behind.
 - Helps prevent non-fast-forward push errors.
-

91. Scenario: Quickly Discard All Local Changes

Problem: You want to reset everything and start fresh.

Solution:

```
git reset --hard HEAD
```

```
git clean -fd
```

Explanation:

- Resets working directory and staging area.
 - `clean -fd` removes untracked files and folders.
-

92. Scenario: Check for Merge Conflicts Before Merging

Problem: You want to test if two branches will conflict.

Solution:

```
git merge --no-commit --no-ff other-branch
```



Check for conflicts

git merge --abort

Explanation:

- Dry-run for a merge. Great for proactive conflict resolution.
-

93. Scenario: Compare a File Across Branches

Problem: You want to compare a file in main vs. dev.

Solution:

git diff main..dev -- path/to/file

Explanation:

- Only shows differences in that specific file between branches.
-

94. Scenario: Clone a Repository into an Existing Directory

Problem: You already have a folder and want to git-ify it.

Solution:

git init

git remote add origin <url>

git fetch

git checkout -t origin/main

Explanation:

- Converts any directory into a working Git clone.
-

95. Scenario: View All Git Configurations

Problem: You're debugging a strange Git behavior.

Solution:

git config --list --show-origin

Explanation:

- Shows values and source files (.gitconfig, .git/config, env).



- Extremely helpful for debugging overrides.
-

96. Scenario: Show Number of Commits Between Two Branches

Problem: You want to count how many commits are ahead/behind.

Solution:

```
git rev-list --count branch1..branch2
```

Explanation:

- Counts commits from one branch not in the other.
 - Used in dashboards or PR stats.
-

97. Scenario: Prevent Merge Commit When Pulling

Problem: Git creates merge commits even for linear histories.

Solution:

```
git config pull.rebase true
```

Explanation:

- Rebase instead of merge for cleaner, linear commit flow.
-

98. Scenario: Enforce Commit Message Standards

Problem: You want every commit message to follow a format (e.g., JIRA-123).

Solution: Use a commit-msg hook:

```
#!/bin/sh
grep -qE '^JIRA-[0-9]+' "$1" || {
    echo "Commit message must start with JIRA-XYZ"
    exit 1
}
```

Explanation:

- Automatically blocks bad commit messages.
- Widely used in enterprise.



99. Scenario: Make a Bare Repository for Central Storage

Problem: You want to host a Git repo but don't need a working copy.

Solution:

```
git init --bare project.git
```

Explanation:

- No working directory.
 - Used on servers or in Git hosting.
-

100. Scenario: Check If a File Has Merge Conflicts

Problem: You're unsure which files are still in conflict.

Solution:

```
git diff --name-only --diff-filter=U
```

Explanation:

- Lists only files with unresolved merge markers.
- Used in merge pipelines and PR workflows.

101. Scenario: Prevent Accidental Push to Main Branch

Problem: You want to avoid pushing directly to main.

Solution:

```
git config branch.main.pushRemote no_push
```

Or use a pre-push hook:

```
#!/bin/sh
```

```
branch=$(git symbolic-ref --short HEAD)
```

```
if [ "$branch" = "main" ]; then
```

```
    echo "Pushing to main is blocked!"
```

```
    exit 1
```

```
fi
```



Explanation:

- Prevents direct pushes.
 - Common in GitOps and CI workflows where main is protected.
-

102. Scenario: List All Files Ever Tracked in Git

Problem: You want to audit every file that's ever existed in the repo.

Solution:

```
git log --pretty=format: --name-only | sort -u
```

Explanation:

- Useful for detecting deleted files or auditing compliance issues.
-

103. Scenario: Find the Commit That Deleted a File

Problem: A file is gone and you want to see who deleted it.

Solution:

```
git log --diff-filter=D --summary | grep delete
```

Explanation:

- Shows deletions in log history.
 - Perfect for accidental deletions or audits.
-

104. Scenario: Enable Autocorrect for Mistyped Git Commands

Problem: You typed git chckout and want Git to help fix typos.

Solution:

```
git config --global help.autocorrect 1
```

Explanation:

- Git will wait briefly and auto-run the closest command.
 - Improves speed for fast typers.
-



105. Scenario: Create a Custom Git Alias

Problem: You frequently use `git log --oneline --graph`.

Solution:

```
git config --global alias.lg "log --oneline --graph --all --decorate"
```

Explanation:

- Aliases simplify frequent commands and improve productivity.
-

106. Scenario: Filter Commits by File Type

Problem: You want to see changes only to `.yaml` files.

Solution:

```
git log -- '**/*.yaml'
```

Explanation:

- Use glob patterns to filter specific types of changes.
-

107. Scenario: Detect Binary Files in Git

Problem: You suspect someone committed large binary files.

Solution:

```
git verify-pack -v .git/objects/pack/*.idx | sort -k3 -n | tail -5
```

Explanation:

- Lists largest objects in Git history.
 - Ideal for performance cleanups.
-

108. Scenario: See All Commits Made on a Specific Date

Problem: You want to audit commits from 2024-12-25.

Solution:

```
git log --since="2024-12-25 00:00" --until="2024-12-25 23:59"
```

Explanation:

- Useful in incident response, code freezes, or sprint reviews.



109. Scenario: Revert Multiple Commits at Once

Problem: A feature was merged with 5 commits and needs to be undone.

Solution:

```
git revert <commit1>^..<commit5>
```

Explanation:

- Reverts all commits in a given range.
 - Maintains clean, traceable revert history.
-

110. Scenario: Change the Author of a Commit

Problem: You committed as the wrong user.

Solution:

```
git commit --amend --author="Correct Name <correct@example.com>"
```

Explanation:

- Use only before push, or with force-push coordination.
-

111. Scenario: Check Which Branches Contain a Commit

Problem: You want to know which branches have abcd1234.

Solution:

```
git branch --contains abcd1234
```

Explanation:

- Useful in feature tracking and bug root cause analysis.
-

112. Scenario: Add a Co-Author to a Commit

Problem: You paired with someone and want to give them credit.

Solution:

```
git commit -m "Fix issue X"
```

Co-authored-by: John Doe <john@example.com>"

Explanation:

- GitHub and GitLab display this clearly in UI.
 - Encourages pairing and accountability.
-

113. Scenario: Apply Only Specific Hunks from a Patch

Problem: You received a .patch file but want to apply parts of it.

Solution:

```
git apply --interactive patch.diff
```

Explanation:

- Interactive application allows selecting hunks.
 - Granular control over patching.
-

114. Scenario: Delete All Local Branches Except Main

Problem: Your workspace is cluttered with old branches.

Solution:

```
git branch | grep -v "main" | xargs git branch -D
```

Explanation:

- Bulk-deletes everything except main.
 - Often used after a big release.
-

115. Scenario: View Repository Size Breakdown

Problem: Repo is slow. You want to check what's taking space.

Solution:

```
git count-objects -vH
```

Explanation:

- Shows number and size of Git objects.
- Helps identify cleanup opportunities.



116. Scenario: Restore Staged File to Working Version

Problem: You want to undo staging but not lose your changes.

Solution:

`git reset HEAD filename`

Explanation:

- Moves it out of staging. Doesn't modify content.
 - Good when you added the wrong file by mistake.
-

117. Scenario: Clone Repo with All Submodules

Problem: You cloned a monorepo but submodules are missing.

Solution:

`git clone --recurse-submodules <repo>`

Explanation:

- Ensures submodules are initialized during clone.
-

118. Scenario: Sync Submodules After Pull

Problem: You pulled latest changes but submodules are outdated.

Solution:

`git submodule update --init --recursive`

Explanation:

- Keeps submodules in sync.
 - Needed in nested module environments.
-

119. Scenario: Disable SSL Verification for Git Temporarily

Problem: You're behind a corporate proxy with SSL issues.

Solution:

`GIT_SSL_NO_VERIFY=true git clone https://repo.git`



Explanation:

- Bypasses SSL check only for that command.
 - Not safe for permanent use.
-

120. Scenario: Automatically Track All New Files in a Folder

Problem: You want Git to always stage new files in /logs/.

Solution:

```
watchman-make -p 'logs/*' --run 'git add logs/ && git commit -m "Auto-track new logs"'
```

Explanation:

- Requires external tool like watchman.
- Useful in logging or sensor data projects.

121. Scenario: Configure Git to Show Colored Output Always

Problem: Git output is dull, and you want better visual cues.

Solution:

```
git config --global color.ui always
```

Explanation:

- Enables color for logs, diffs, and status.
 - Boosts visibility in large output and CI logs.
-

122. Scenario: Show Only Merge Commits in Log

Problem: You want to audit merge activity across branches.

Solution:

```
git log --merges --oneline
```

Explanation:

- Filters out all commits except merges.
 - Excellent for reviewing integration points and PR history.
-

123. Scenario: List Unpushed Commits on Current Branch

Problem: Before pushing, you want to see what's new.

Solution:

```
git log origin/${git branch --show-current}..HEAD
```

Explanation:

- Compares local branch to its remote counterpart.
 - Prevents accidental deployment of WIP code.
-

124. Scenario: Find Commits With Specific Message Pattern

Problem: You want to list all commits mentioning HOTFIX.

Solution:

```
git log --grep=HOTFIX
```

Explanation:

- Useful for changelogs, compliance, or backtracking incident fixes.
-

125. Scenario: Shallow Clone a Repo with History Depth

Problem: You want limited history for performance.

Solution:

```
git clone --depth 5 <repo-url>
```

Explanation:

- Speeds up clone by fetching last 5 commits only.
 - Saves bandwidth on CI agents or ephemeral environments.
-

126. Scenario: Prevent Pulling Binary File Changes

Problem: You're on slow internet and don't need .zip updates.

Solution:

Use sparse checkout and LFS filtering:

```
git lfs track "*.zip"
```

```
echo "*.zip filter=lfs diff=lfs merge=lfs -text" > .gitattributes
```



Explanation:

- Prevents large files from syncing unless needed.
-

127. Scenario: Create Git Hook to Prevent Debug Code

Problem: You want to block commits containing console.log.

Solution (pre-commit hook):

```
#!/bin/sh

if git diff --cached | grep -i "console.log"; then
    echo "Remove console.log before committing!"
    exit 1
fi
```

Explanation:

- Catches debug remnants early in the workflow.
 - Especially valuable in frontend teams.
-

128. Scenario: Get Total Number of Commits in Repo

Problem: You want project statistics.

Solution:

```
git rev-list --all --count
```

Explanation:

- Helps estimate codebase activity and age.
-

129. Scenario: Skip Hooks When Committing

Problem: You're debugging and want to bypass commit hooks.

Solution:

```
git commit --no-verify
```

Explanation:

- Temporary override. Don't use it as a habit.



- Useful when the hook logic is buggy.
-

130. Scenario: Reset Remote URL from HTTPS to SSH

Problem: Git keeps asking for credentials.

Solution:

```
git remote set-url origin git@github.com:user/repo.git
```

Explanation:

- Switches protocol from HTTPS to SSH.
 - Recommended for daily developers.
-

131. Scenario: Sync Your Forked Branch with Upstream main

Problem: Your main is outdated, and you want upstream updates.

Solution:

```
git fetch upstream
```

```
git checkout main
```

```
git merge upstream/main
```

```
git push origin main
```

Explanation:

- Maintains alignment with parent repository.
 - Critical for contributing via PRs.
-

132. Scenario: Get the SHA of the Latest Commit on a Branch

Problem: You want the latest commit ID for CI.

Solution:

```
git rev-parse main
```

Explanation:

- Returns clean SHA.
- Ideal for scripting and tagging automation.



133. Scenario: Rebase One Branch onto Another With Conflicts

Problem: You want to update a feature branch cleanly onto main.

Solution:

`git checkout feature`

`git rebase main`

Explanation:

- Applies each feature commit on top of main.
 - If conflicts arise, Git pauses and prompts for resolution.
-

134. Scenario: Archive Your Git Project as a Zip File

Problem: You want to share your project without .git.

Solution:

`git archive -o code.zip HEAD`

Explanation:

- Creates a clean archive of working directory.
 - Good for handovers or deployments.
-

135. Scenario: Compare Working Tree with Last Commit

Problem: You want to know what you've changed since last commit.

Solution:

`git diff HEAD`

Explanation:

- Compares current state with the last commit.
 - Very common in daily Git usage.
-

136. Scenario: Compare Staging Area to Last Commit

Problem: You want to validate staged changes only.



Solution:

```
git diff --cached
```

Explanation:

- Shows only what is staged for the next commit.
-

137. Scenario: Show All Remote URLs

Problem: You forgot which remotes are set.

Solution:

```
git remote -v
```

Explanation:

- Lists fetch and push URLs.
 - Helps avoid mistakes when pushing to multiple remotes.
-

138. Scenario: Convert an Existing Folder to a Git Repo

Problem: You built a project manually and now want version control.

Solution:

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

Explanation:

- Initializes Git and captures current state.
-

139. Scenario: Delete a Tag Locally and Remotely

Problem: A tag was pushed with the wrong version.

Solution:

```
git tag -d v1.2
```

```
git push origin :refs/tags/v1.2
```

Explanation:

- Local delete + remote delete.
 - Always inform team when doing this post-release.
-

140. Scenario: Find When a File Was Renamed

Problem: You think a file was renamed and want to verify.

Solution:

```
git log --follow filename
```

Explanation:

- Tracks history across renames.
- Essential for legacy project debugging.

141. Scenario: Force Push a Specific Branch Only

Problem: You need to force push feature-x but leave others untouched.

Solution:

```
git push origin feature-x --force-with-lease
```

Explanation:

- Safer than --force; it refuses to push if the remote was updated behind your back.
 - Used when rebasing or squashing commits on a feature branch.
-

142. Scenario: View Commits That Touched a Function

Problem: You want to know who changed a specific function in a file.

Solution:

```
git log -L :function_name:file.py
```

Explanation:

- Requires Perl-compatible regex and Git built with libpcre.
 - Deeply useful in large codebases to audit function evolution.
-

143. Scenario: Create a Branch That Matches a Tag

Problem: You want to start working on a hotfix from a release tag.



Solution:

```
git checkout -b hotfix-v1.2 v1.2
```

Explanation:

- Branches from the exact snapshot tagged v1.2.
 - Helps in patching or point releases.
-

144. Scenario: Enforce Line Endings Across a Repo

Problem: You're getting mixed LF/CRLF issues on different OSes.

Solution:

```
echo "* text=auto" > .gitattributes
```

```
git add .gitattributes
```

```
git commit -m "Enforce consistent line endings"
```

Explanation:

- Ensures cross-platform consistency.
 - Avoids problems in CI/CD pipelines.
-

145. Scenario: Restore Deleted Untracked Files

Problem: You used `git clean -fd` and lost important local files.

Solution:

If you don't have a backup or stash — **you can't recover.**

Best Practice:

Before cleanup, run:

```
git clean -nfd
```

Explanation:

- `-n` does a dry run and shows what will be deleted.
 - A safety net you should always use.
-

146. Scenario: Lock Branch from Pushing by Mistake

Problem: You want to prevent even yourself from pushing to main.



Solution:

Add this in .git/config:

```
[remote "origin"]
```

```
push = +refs/heads/develop:refs/heads/develop
```

Explanation:

- Limits what branches are allowed to push.
 - Add pre-push hook too for double safety.
-

147. Scenario: Commit Changes From Multiple Authors

Problem: Two people worked on a file and both need credit.

Solution:

```
git commit -m "Improved parser
```

```
Co-authored-by: Alice <alice@example.com>
```

```
Co-authored-by: Bob <bob@example.com>"
```

Explanation:

- GitHub/GitLab recognizes this format.
 - Keeps authorship transparent.
-

148. Scenario: Clone With Specific Branch and Depth

Problem: You want branch qa with only the last 3 commits.

Solution:

```
git clone --branch qa --single-branch --depth 3 <url>
```

Explanation:

- Efficient cloning, ideal for CI runners or limited bandwidth.
-

149. Scenario: Save Git Credentials Temporarily

Problem: Git asks for credentials repeatedly during operations.



Solution:

```
git config --global credential.helper cache
```

Explanation:

- Stores credentials temporarily in memory (~15 mins).
 - Use store instead of cache for disk-based (less secure).
-

150. Scenario: Filter All Commits by a Specific File

Problem: You want a list of commits that touched docker-compose.yml.

Solution:

```
git log -- docker-compose.yml
```

Explanation:

- Tracks only the file's history.
 - Use with --stat or --patch for more details.
-

151. Scenario: Quickly Stage All Modified Files

Problem: You modified several files and want to stage all changes.

Solution:

```
git add -u
```

Explanation:

- Stages only modified and deleted files (not new untracked ones).
-

152. Scenario: Show Who Created a Branch

Problem: You're auditing and want to know who pushed feature-abc.

Solution:

Not tracked directly by Git.

Workaround:

Use server-side Git hooks or audit logs (GitHub, GitLab).

Explanation:

- Git doesn't store branch creator info locally.



- Enforce accountability via protected branches and reviews.
-

153. Scenario: Automatically Add File Headers to New Files

Problem: You want all new .py files to include a license header.

Solution:

Use a pre-commit hook:

```
#!/bin/sh
```

```
for file in $(git diff --cached --name-only --diff-filter=A | grep '.py$'); do
```

```
  cat header.txt "$file" > temp && mv temp "$file"
```

```
  git add "$file"
```

```
done
```

Explanation:

- Automates boilerplate and enforces policy.
 - Works across file types with customization.
-

154. Scenario: Remove File From Git but Keep in Working Tree

Problem: You want Git to forget about debug.log.

Solution:

```
git rm --cached debug.log
```

```
echo "debug.log" >> .gitignore
```

```
git commit -m "Untrack debug.log"
```

Explanation:

- Prevents accidental re-additions.
-

155. Scenario: Keep File in History but Ignore Future Changes

Problem: You want to track config.yaml once and ignore all future changes.



Solution:

```
git update-index --assume-unchanged config.yaml
```

Explanation:

- Git ignores local changes unless overridden.

To undo:

```
git update-index --no-assume-unchanged config.yaml
```

156. Scenario: Roll Back a Specific File to a Given Commit

Problem: Only one file needs reverting to its state in abc123.

Solution:

```
git checkout abc123 -- path/to/file
```

Explanation:

- Replaces your working version.
 - Doesn't alter commit history until you stage and commit.
-

157. Scenario: Cherry-Pick Multiple Commits Together

Problem: You want to bring over 3 non-sequential commits.

Solution:

```
git cherry-pick <sha1> <sha2> <sha3>
```

Explanation:

- Applies each commit in the given order.
 - Useful for reusing fixes or backporting.
-

158. Scenario: Audit Who Changed a File the Most

Problem: You want ownership stats for main.py.

Solution:

```
git blame main.py | cut -d '(' -f 2 | cut -d ' ' -f 1 | sort | uniq -c | sort -nr
```

Explanation:

- Tells you which contributor made the most changes to a file.
 - Valuable in code ownership reviews.
-

159. Scenario: Find the First Commit Ever Made

Problem: You want to trace the project's origin.

Solution:

```
git rev-list --max-parents=0 HEAD
```

Explanation:

- Returns the root commit SHA.
-

160. Scenario: Pause a Rebase to Manually Test Code

Problem: During interactive rebase, you want to stop midway to run tests.

Solution:

In rebase editor:

```
edit <commit-sha>
```

Then manually test, modify, and:

```
git rebase --continue
```

Explanation:

- Ideal for testing refactorings during rebase.

161. Scenario: Prevent Large Files From Being Committed

Problem: Someone accidentally committed a 300MB .log file.

Solution: Use a pre-commit hook:

```
#!/bin/sh
```

```
maxsize=5242880
```

```
for file in $(git diff --cached --name-only); do
```

```
if [ -f "$file" ] && [ $(stat -c%s "$file") -gt $maxsize ]; then
```

```
echo "File $file exceeds 5MB limit. Commit aborted."
```



```
exit 1
```

```
fi
```

```
done
```

Explanation:

- Blocks large files early.
 - Use Git LFS for valid large binaries.
-

162. Scenario: Enforce Commit Message Format via Hook

Problem: All commits must start with [ISSUE-123] format.

Solution: commit-msg hook:

```
#!/bin/sh
msg=$(cat "$1")
if ! echo "$msg" | grep -qE "^[ISSUE-[0-9]+\]"; then
    echo "Invalid commit message format"
    exit 1
fi
```

Explanation:

- Validates messages before allowing commit.
 - Enhances traceability in large teams.
-

163. Scenario: Automatically Rebase When Pulling

Problem: You want to avoid merge commits after every pull.

Solution:

```
git config --global pull.rebase true
```

Explanation:

- Rebase creates a linear history.
 - Used in cleaner Git strategies like GitFlow.
-



164. Scenario: Update a Commit Message Deep in History

Problem: A commit 5 steps back has a typo in its message.

Solution:

```
git rebase -i HEAD~5
```

```
# Change 'pick' to 'reword' for the desired commit
```

Explanation:

- Lets you edit the message during interactive rebase.
 - Should be done before push or with force-push caution.
-

165. Scenario: Detect Duplicate Commits (Same Content)

Problem: You suspect duplicate changes were committed twice.

Solution:

```
git log --pretty=format:"%h %s" | sort | uniq -d
```

Explanation:

- Finds duplicate subjects.
 - Combine with `git show <sha>` for deeper inspection.
-

166. Scenario: View All Branches That Are Ahead of Main

Problem: You want to know what features haven't been merged yet.

Solution:

```
for branch in $(git branch --format='%(refname:short)'); do  
  git log --oneline main..$branch | grep . > /dev/null && echo "$branch"  
done
```

Explanation:

- Lists branches with unmerged commits compared to main.
-

167. Scenario: Restore the .gitignore from a Previous Commit

Problem: .gitignore was accidentally wiped.



Solution:

```
git checkout <good-commit-hash> -- .gitignore
```

Explanation:

- Pulls the specific file's state from history.
-

168. Scenario: Create a Clean Branch With Just One File

Problem: You want to create a new repo with only README.md.

Solution:

```
git checkout --orphan clean-branch
```

```
git rm -rf .
```

```
git checkout main -- README.md
```

```
git commit -m "Init with README only"
```

Explanation:

- Starts a fresh history with selective files.
 - Common in documentation sub-repos.
-

169. Scenario: Sync All Tags Between Local and Remote

Problem: You have local tags not pushed.

Solution:

```
git push --tags
```

Explanation:

- Ensures tag consistency across devs and pipelines.
-

170. Scenario: Clean Unreferenced Git Objects

Problem: You rewrote history and want to garbage collect.

Solution:

```
git gc --prune=now
```

Explanation:

- Cleans up orphaned objects.
 - Speeds up repository.
-

171. Scenario: Audit Commits on a Specific Date by Author

Problem: You want to know what Alice committed on 2025-01-01.

Solution:

```
git log --author="Alice" --since="2025-01-01" --until="2025-01-01 23:59"
```

Explanation:

- Targeted audit, useful in incident timelines.
-

172. Scenario: Clone Without History or .git Folder

Problem: You just want the latest code snapshot.

Solution:

```
git archive --remote=<repo-url> HEAD | tar -x
```

Explanation:

- Retrieves working directory only.
 - Useful in deployments.
-

173. Scenario: Identify Reflog Expiry Time

Problem: You want to understand when reflog entries expire.

Solution:

```
git config --show-origin --get-all gc.reflogExpire
```

Explanation:

- Defaults to 90 days.
 - Tune via `git config --global gc.reflogExpire "30 days"`.
-

174. Scenario: Stop Rebase After a Specific Commit

Problem: You want to rebase up to a commit and not beyond.



Solution:

```
git rebase -i <base-sha>
```

```
# In the editor, replace all commits after target with `drop`
```

Explanation:

- Gives precise rebase control.
-

175. Scenario: Remove All Empty Commits

Problem: Your log is cluttered with no-op commits.

Solution:

```
git rebase -i --root
```

```
# Delete or squash commits with no changes
```

Explanation:

- Requires caution if already pushed.
 - Helps clean noisy logs.
-

176. Scenario: Create Branch From Another Remote

Problem: You want to track feature-x from upstream.

Solution:

```
git checkout -b upstream-feature upstream/feature-x
```

Explanation:

- Useful in forked repos when working on non-main branches.
-

177. Scenario: Remove Credential Helper Setting

Problem: You want Git to stop saving passwords.

Solution:

```
git config --global --unset credential.helper
```

Explanation:

- Prevents Git from remembering creds between sessions.



178. Scenario: Force Reapply All Commits on a Branch

Problem: You want to rebuild all commits atop the same base (e.g., change author or timestamps).

Solution:

```
git rebase --root -i
```

Explanation:

- Applies all commits from initial commit forward.
-

179. Scenario: Split a Commit into Multiple Commits

Problem: One commit has unrelated changes.

Solution:

```
git reset -soft HEAD~1
```

```
# Use `git add -p` to stage selectively and commit step-by-step
```

Explanation:

- Allows surgical separation of changes.
-

180. Scenario: See Diff Stats by Author

Problem: You want to see how many lines each contributor added/removed.

Solution:

```
git log --author="Alice" --pretty=tformat: --numstat | awk '{ add += $1; del += $2 } END { print "Added:", add, "Deleted:", del }'
```

Explanation:

- Great for analytics or review tracking.

181. Scenario: Track Changes by File Extension

Problem: You want to see all .js commit diffs in the project.

Solution:

```
git log -- '*.js'
```



Explanation:

- Uses pathspec to filter logs.
 - You can add --stat or --patch for full diffs.
-

182. Scenario: Clone Repo But Exclude Large Folders

Problem: You want to exclude assets/ from the clone to save space.

Solution:

```
git clone --filter=blob:none --sparse <repo>
```

```
cd <repo>
```

```
git sparse-checkout set src/ docs/
```

Explanation:

- Clone only specific folders.
 - Ideal for partial contributions.
-

183. Scenario: Convert Mixed Repositories to Monorepo

Problem: You want to merge multiple project repos into one.

Solution:

```
git remote add proj2 <url>
```

```
git fetch proj2
```

```
git merge --allow-unrelated-histories proj2/main
```

Explanation:

- Preserves commit histories.
 - Used in corporate monorepo migration plans.
-

184. Scenario: Fix Git "index.lock" Error

Problem: Git says "unable to create 'index.lock' file."



Solution:

```
rm -f .git/index.lock
```

Explanation:

- Happens when a Git process is interrupted.
 - Ensure no other Git ops are running.
-

185. Scenario: Completely Reset Local Repo to Remote

Problem: Your repo is a mess — you want to force re-clone it.

Solution:

```
git fetch origin
```

```
git reset --hard origin/main
```

```
git clean -fd
```

Explanation:

- Re-syncs everything to the current remote state.
-

186. Scenario: Diff Between Two Tags

Problem: You want to know what changed between v1.0 and v2.0.

Solution:

```
git diff v1.0 v2.0
```

Explanation:

- Can add --stat to show only file-level summary.
-

187. Scenario: Show Time Taken Between Two Commits

Problem: You want to track duration of a feature branch.

Solution:

```
git log --format='%ai %s' <start>..<end>
```

Explanation:

- Shows timestamps for all commits between two points.



188. Scenario: Track Deleted Files Over Time

Problem: You want a list of files that were deleted in history.

Solution:

```
git log --diff-filter=D --summary | grep delete
```

Explanation:

- Helps recover removed files or audit what was removed.
-

189. Scenario: Find Commits by Keyword in Code (Not Message)

Problem: You want to find all commits where someone added console.log.

Solution:

```
git log -S"console.log"
```

Explanation:

- Searches for code string diffs instead of commit messages.
 - Very powerful debugging tool.
-

190. Scenario: Temporarily Disable Git Hooks

Problem: You're debugging hooks and want to skip them.

Solution:

```
GIT_PARAMS_SKIP_HOOKS=true git commit --no-verify
```

Explanation:

- Environment variable + --no-verify lets you override strict environments.
-

191. Scenario: Rewrite History to Replace an Email Globally

Problem: You used your personal email and want to fix it across all commits.

Solution:

```
git filter-branch --commit-filter '
```

```
if [ "$GIT_AUTHOR_EMAIL" = "old@example.com" ];
```



then

```
GIT_AUTHOR_NAME="New Name";
```

```
GIT_AUTHOR_EMAIL="new@example.com";
```

```
git commit-tree "$@";
```

else

```
git commit-tree "$@";
```

fi' HEAD

Explanation:

- Full historical rewrite.
- Use git filter-repo for large repos.

192. Scenario: Automate Version Bumps Based on Tags

Problem: You want to create semantic versioning with each release.

Solution:

```
latest=$(git describe --tags --abbrev=0)
```

```
next="v$(echo $latest | awk -F. '{ $3 += 1; print $1 "." $2 "." $3 }')
```

```
git tag $next
```

```
git push origin $next
```

Explanation:

- Automates patch version incrementing.

193. Scenario: Show Diffs for Only Renamed Files

Problem: You renamed files and want to verify only those changes.

Solution:

```
git diff --diff-filter=R --summary
```

Explanation:

- R stands for rename.
- Helps confirm mass file restructures.



194. Scenario: Stop Tracking File Renames in Git Diff

Problem: You want to treat renamed files as deleted + added.

Solution:

```
git diff --no-renames
```

Explanation:

- Treats moved files as separate changes.
 - Used in changelog generation or code review tooling.
-

195. Scenario: Undo a Single File From a Commit

Problem: You committed multiple files, but want to undo only one.

Solution:

```
git checkout HEAD^ -- file.txt
```

```
git commit --amend
```

Explanation:

- Replaces file with previous version, then amends the commit.
-

196. Scenario: Create Signed Tags for Releases

Problem: You want to cryptographically sign release tags.

Solution:

```
git tag -s v1.0 -m "Signed release"
```

Explanation:

- Requires GPG configured.
 - Adds trust and verifiability.
-

197. Scenario: Remove All Commits From Git History

Problem: You want to start fresh but keep the current working directory.



Solution:

```
rm -rf .git  
git init  
git add .  
git commit -m "Fresh start"
```

Explanation:

- Destroys history and starts from scratch.
-

198. Scenario: Undo a Remote Push

Problem: You force pushed the wrong branch.

Solution:

```
# Find previous ref  
git reflog  
git reset --hard <previous-sha>  
git push --force
```

Explanation:

- Use only if you're confident and your team is aware.
-

199. Scenario: Track the Last Commit Per File

Problem: You want a list of files with their last commit message.

Solution:

```
git ls-tree -r HEAD --name-only | while read file; do  
    echo -n "$file: "  
    git log -1 --pretty=format:"%h %an %s" -- "$file"  
    echo  
done
```

Explanation:

- Outputs last commit affecting each file — useful for audits.



200. Scenario: Find Time Between Commit and Push

Problem: You want to analyze delays between authoring and pushing.

Solution:

- Git alone doesn't store push times.
- Use GitHub API or GitLab CI/CD pipelines for timestamps.

Explanation:

- For detailed audits, integrate Git server APIs with analytics tools.